CloudGenie Library Agents

Architecture and Operations Guide

Version 1.0 | 10 November 2026

This framework helps you reduce the number of files you need to navigate while retaining a complete record of the originals. A coordinator assigns yearly document, photo and hash workers, then combines their evidence across the selected library.

Start with an extracted Google Takeout copy. The scan builds an inventory, separates exact duplicates from similar items, and creates subject reading packs with references back to their sources. No scan command changes original data.

Cleanup is an independent reviewed workflow. Exact-byte copies may be quarantined only after approval and fresh verification. Permanent purge requires another approval and typed confirmation. Pixel similarity, topic grouping and AI summaries cannot authorize deletion.

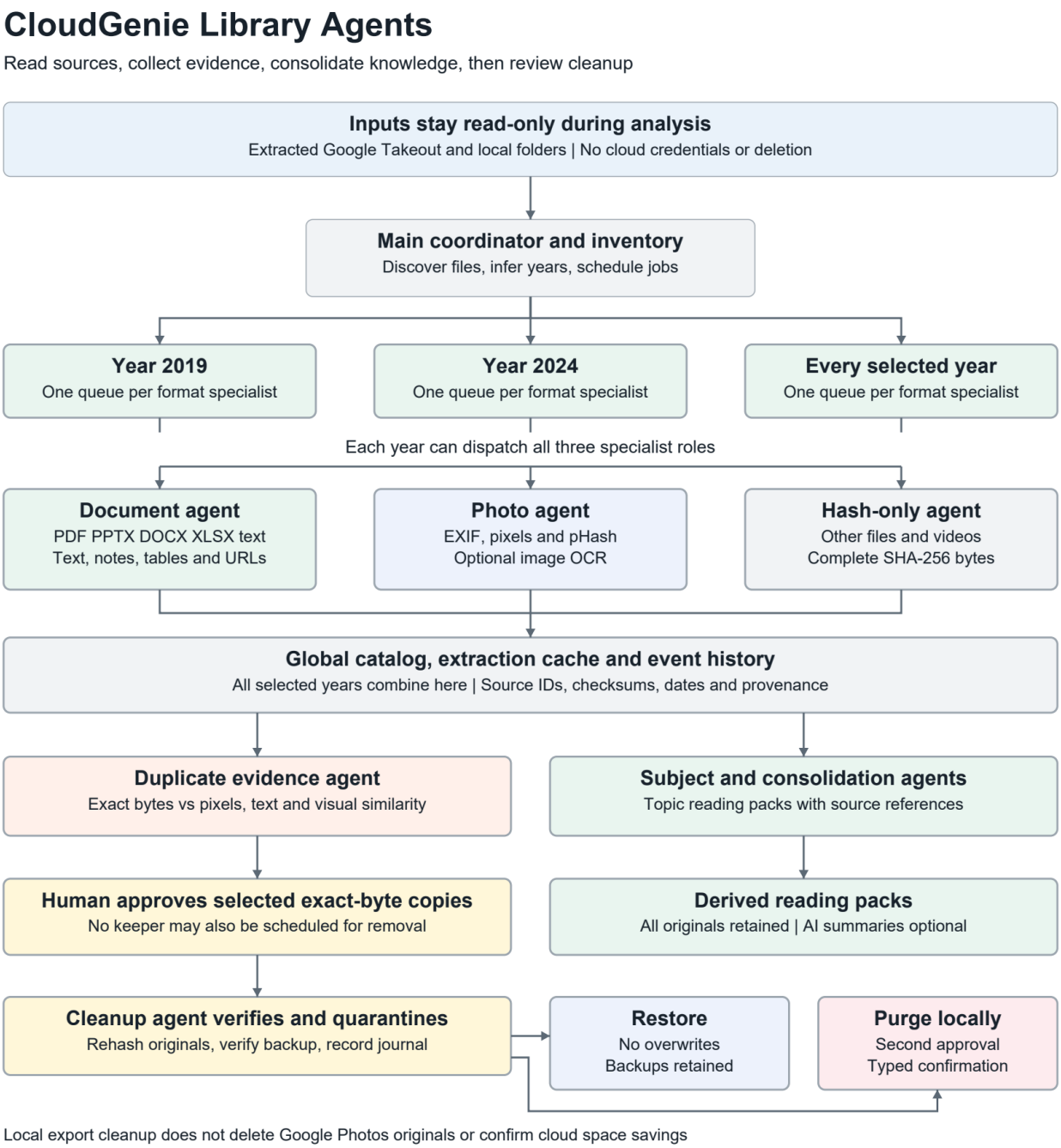
MCP is optional. The application runs directly on Windows, WSL Ubuntu or a container. Google Photos cloud deletion is not implemented; local export cleanup does not free cloud quota.

How to use this guide

Review the architecture illustration, follow the installation and demo instructions, and inspect the generated inventory before using any cleanup commands. The operations section covers approval, quarantine, restore and optional purge. The technical appendix documents worker contracts, data structures and scaling limits.

The accompanying Markdown files contain copyable commands and editable Mermaid diagram source. Source files and tests are included in the release package.

Architecture Illustration



Year workers all contribute to the shared catalog. Similarity and consolidation produce review evidence; only a separately approved exact-byte plan reaches local cleanup. See the full-resolution PNG, SVG and PDF for an enlarged view.

A Python coordinator and specialist workers for organizing a personal document and photo library. Run it on Windows, WSL Ubuntu, Linux, or a container with access to your exported files. The default scan is read-only and works without an LLM, MCP, Google credentials, or a paid API.

The workers inventory each year, extract document content, compare duplicate evidence across all years, and assemble subject reading packs. Cleanup is a separate command sequence: review an exact-byte plan, quarantine approved copies, then optionally approve a separate permanent purge.

What is included

- One coordinator and inventory worker; dynamically scheduled year and format specialist jobs.
- Text, PDF, PPTX, DOCX and XLSX extraction, including slide notes, document tables, formula strings and document hyperlinks where supported.
- Complete SHA-256 file hashing, decoded-image matches and perceptual photo candidates.
- Global matching across years and multiple source folders.
- Takeout JSON sidecar association, capture-year inference and recognition of repeated export representations of a single Photos item.
- Topic suggestions using TF-IDF and MiniBatchKMeans, followed by combined Markdown reading packs with per-block source references.
- Optional image OCR and optional summaries through a local Ollama service.
- SQLite cache, JSON event history, CSV inventory, JSON evidence and explicit extraction errors.
- Reviewed quarantine, restore, and separately approved permanent purge of local copies.
- A Dockerfile, disposable demo, regression tests, illustrated architecture and a printable operations guide.

These are task-specific software agents with deterministic permissions and scheduling. They are not independent LLM conversations and are not a multi-machine cluster. The optional local model writes an unverified summary only; it cannot call tools, change an approval, or delete a file.

Install on WSL Ubuntu

Use Python 3.10 or newer. From this app directory:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[analysis]'
python -m library_agents doctor
```

On Ubuntu, install `python3-venv` if the venv command reports that it is missing. Windows drives are available under `/mnt/c`, `/mnt/d`, and so on. Extract every Takeout ZIP part into the same independent export folder before scanning; archive contents are not traversed by this app.

Install on Windows PowerShell

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e '.[analysis]'
.\.venv\Scripts\python.exe -m library_agents doctor
```

The commands below use `python`. In PowerShell, substitute `.\.venv\Scripts\python.exe` unless that virtual environment is already activated.

Try disposable demo data first

```
python examples/create_demo.py demo-data
python -m library_agents scan --source demo-data --output runs/demo
```

The demo intentionally contains exact duplicate text and image files, related documents, and an image with identical pixels but different metadata. Existing demo directories are never overwritten. The example generator also creates a PDF when ReportLab is installed. Optional fixture formats are skipped if their packages are unavailable.

Scan your exports

```
python -m library_agents scan \
  --source '/mnt/d/GoogleExport/Takeout/Google Photos' \
  --source '/mnt/d/GoogleExport/Takeout/Drive' \
  --output '/mnt/d/LibraryAudit/run-001' \
  --workers 4
```

PowerShell equivalent:

```
python -m library_agents scan --source 'D:\GoogleExport\Takeout\Google Photos' --source
   'D:\GoogleExport\Takeout\Drive' --output 'D:\LibraryAudit\run-001' --workers 4
```

Sources and run output must be separate directories; neither may contain the other. Symlinks and directory junctions are skipped or rejected. Do not edit source files while an audit or cleanup is running. Prefer an independent export folder rather than a live Drive or OneDrive sync folder, where local cleanup could propagate through that sync service.

To limit an investigation:

```
python -m library_agents scan --source /data/export --output /data/audit-2024 --year
  2024 --extensions pdf,pptx,docx,txt,jpg,png
```

A filtered scan cannot establish whole-library uniqueness. Excluded items and filter reasons are recorded. Sidecar dates take priority, then image EXIF dates, a year in the filename or folder, then filesystem modification time in UTC. Modification time can be the download date. A named year worker owns one year and one specialist queue; the duplicate and subject stages still compare the entire selected corpus.

You may also edit `config.example.json` for your machine and run:

```
python -m library_agents scan --config config.example.json
```

Command-line values override corresponding config settings. Config source paths are interpreted relative to the working directory, not the config file. Never put account passwords or tokens in this configuration; no cloud authentication is used.

Inspect the outputs

File	Purpose
`REPORT.md`	Overview, coverage, limitations and next steps
`summary.json`	Counts, years, categories, cache use and agent timings
`catalog.json`	Authoritative file IDs, paths, hashes, dates, sidecars and extracted text
`inventory.csv`	A sortable per-file inventory
`duplicates.json`	Exact bytes, export repeats, matching pixels, matching text and similar photos
`topics.json`	Suggested subjects and their source IDs
`consolidated/<run-id>/`	Current run's combined topic packs and source maps
`urls.csv`	Links found in extracted document content and supported relationships
`cloud-review.csv`	Original Photos links where available; every decision starts pending
`events.jsonl`	Worker scheduling and completion events
`cache.sqlite3`	Local extraction cache for resumable scans
`status.json`	Running or completed state

Outputs can contain private extracted text and file names. Store them privately. The catalog's content hash binds each approval plan to that exact inventory. Rescanning replaces the catalog and invalidates earlier plans; earlier topic packs live in their own run-ID directory. Do not rescan a run directory after quarantining files if you need that catalog for restoration. Start a new run directory for a new inventory instead.

Understand the evidence

Evidence	Meaning	Cleanup eligibility
Complete SHA-256 and size match	Same file bytes, revalidated before mutation	May be proposed; explicit approval still required
Same hash and same Photos cloud URL	Repeated representation of one item in export folders	Excluded from generated cleanup plans
Same decoded pixels, different bytes	Image data appears equal but metadata or encoding differs	Review only
Perceptual hash near-match	Visually similar image candidate	Review only
Same normalized extracted text	Text agrees; media, notes, signatures or other data may differ	Review only
Same suggested topic	Related content worth reading together	Never a deletion reason

Hardlink aliases do not become cleanup actions. GIFs, multipage TIFFs and other multiframe images get complete byte comparisons only. Low-information images skip perceptual comparison. Image pHash candidates use up to five nearest neighbors and are not exhaustive, particularly for crops. Exact matching is global and has no similarity-item cap; similarity and subject stages are explicitly skipped above the configured limit.

Consolidate related content

The subject worker suggests groups from extracted document text. The consolidation worker produces one reading pack per suggested topic, collapses repeated text blocks, and lists every contributing file ID and page, slide or block reference. It retains all originals. This is a study and navigation artifact, not a lossless merged PDF, PowerPoint or workbook.

Do not delete original documents merely because a pack exists: it may omit figures, editable elements, comments, tracked revisions, spreadsheet formatting, embedded objects or layout. Truncated and failed extractions remain marked in the catalog. Topic grouping is statistical and can put unrelated content together; review the group and source map before relying on it.

For an optional local-model summary, first install and run Ollama with a model of your choice. Then supply its installed model name:

```
python -m library_agents scan --source /data/export --output /data/audit-with-summary
  --llm-model YOUR_INSTALLED_MODEL
```

Only this explicit flag sends source text to `http://127.0.0.1:11434/api/chat`. Each request uses at most the first 50,000 characters of a pack. The response is written to a separate `AI-SUMMARY.md` file and labeled unverified. No cloud model or model download is configured by this project. In a container, loopback refers to that container; the optional model must be reachable there through an explicitly configured environment, otherwise the app writes an error artifact and retains the source pack.

Optional OCR applies to image files only. Install the OCR extra and the Tesseract executable separately, then use `--ocr`. Scanned PDFs are marked for OCR; this version does not render PDF pages for OCR.

```
python -m pip install -e '.[analysis,ocr]'
python -m library_agents scan --source /data/export --output /data/audit-ocr --ocr
```

Review and quarantine local duplicates

Generate an unapproved plan only after reviewing the audit:

```
python -m library_agents plan --run /data/audit --output /data/quarantine-plan.json
```

Review each keeper and proposed removal. Change only the selected actions' `approved` booleans from `false` to `true`. Keep file IDs, displayed paths, hashes, run ID and catalog hash unchanged. Unapproved actions are ignored.

Always preview the reviewed plan:

```
python -m library_agents apply --run /data/audit --plan /data/quarantine-plan.json
  --quarantine /data/quarantine
```

To execute that reviewed plan, add `--execute`:

```
python -m library_agents apply --run /data/audit --plan /data/quarantine-plan.json
  --quarantine /data/quarantine --execute
```

The worker preflights every approved pair and rehashes retained copies. It creates and verifies a separate quarantine copy, revalidates the source and retained copy, then removes the approved local source. A journal records each operation. A changed file, edited plan, missing keeper, hardlink alias or conflicting destination blocks the operation. Quarantine must be outside every source and the run folder. Quarantine initially retains the bytes, so it does not itself reclaim all local storage.

The plan cannot schedule any retained copy for removal. Execution is sequential after concurrent analysis. Keep the machine and source files stable during execution. Operations are journaled per file, not a whole-library filesystem transaction; an I/O failure can leave earlier operations completed, with verified backups available for restoration.

Restore quarantined copies

Preview first, then execute when appropriate:

```
python -m library_agents restore --run /data/audit --quarantine /data/quarantine
python -m library_agents restore --run /data/audit --quarantine /data/quarantine
  --execute
```

Restore never overwrites an existing different file. It verifies the backup hash and retains the quarantine backup after restoration. Keep the original run's catalog and quarantine journal together. A permanently purged copy cannot be restored by this tool.

Optional permanent purge

Permanent deletion is deliberately separate from quarantine. Create another plan:

```
python -m library_agents purge-plan --run /data/audit --quarantine /data/quarantine
  --output /data/purge-plan.json
```

Review it independently and approve only selected actions. Preview:

```
python -m library_agents purge --run /data/audit --quarantine /data/quarantine --plan
  /data/purge-plan.json
```

Only an explicit permanent-deletion command removes verified quarantine copies:

```
python -m library_agents purge --run /data/audit --quarantine /data/quarantine --plan
  /data/purge-plan.json --execute --confirm PERMANENTLY_DELETE_APPROVED_QUARANTINE
```

The purge worker checks that each copy is journaled as quarantined and that its retained original still matches the full hash. It never purges by similar appearance, same text, or a topic suggestion. The implementation does not defend against an administrator maliciously replacing files during the final filesystem checks; do not run concurrent writers against the library.

Run in Docker or a sandbox

Build from this app directory:

```
docker build -t library-agents .
mkdir -p /data/library-audit
docker run --rm --network none --user "$(id -u):$(id -g)" \
  -v /data/export:/input:ro \
  -v /data/library-audit:/output \
  library-agents scan --source /input --output /output/run-001
```

The image installs analysis dependencies during build; the scan itself needs no network. The container runs as a non-root user, with `--user` matching your Ubuntu user so bind-mounted output remains writable. Adjust host paths to your machine. A generic sandbox must similarly receive the input folder and a persistent output mount.

Catalog paths belong to the environment that created them. `/input/...` inside a container is not a Windows path. Run later cleanup or restore in the same path namespace, or create a fresh audit on the host where you intend to execute it. Audit containers should retain read-only source mounts; do not grant write access merely to test the analysis.

Google Photos and MCP

MCP is not required. Python workers directly read the local files they are authorized to inspect. MCP can later expose selected reports or explicit tools to an assistant, but it does not add data permissions, bypass Google's API restrictions, or make deletion reliable.

Google's Photos Library API is restricted to app-created media. It cannot provide this app a complete inventory of a pre-existing personal library. This implementation consumes your Takeout export and produces original-cloud review links when the sidecars supply them. It does not scrape private Photos endpoints, use browser cookies, authenticate to Google, or delete cloud items.

Deleting a local export copy does not delete the Google Photos original or free its cloud quota. Conversely, a repeated album representation can correspond to only one cloud item. Photos cleanup must remain a separate, reviewed cloud operation with confirmed item identities and account context.

An optional future MCP adapter should expose read-only `get_status`, `get_summary` and `list_review_candidates` first. Keep source roots configured by the operator, avoid arbitrary path tools, and do not expose permanent purge as an automatically callable agent tool. That adapter is an architectural extension, not included executable server code.

Operational limits and recovery

- This version schedules local threads. It does not distribute work to remote machines, use a message broker, or provide an interactive web dashboard.
- The first run hashes every selected file in full and therefore reads the entire selected byte volume. Start with 2-4 workers on a laptop; additional concurrency may slow a single disk.
- All selected file records and extracted text are held in memory. Default extraction is capped at 500,000 characters per document. For large corpora, lower that cap and work in verified batches; global cross-batch duplicate aggregation is not included.
- Default similarity limit is 10,000 eligible items per modality. Nearest-neighbor comparison is batched but still has substantial pairwise compute cost. Increase the limit only after measuring on a subset.

- The cache key includes path, size, modification and change times, filesystem identity, settings and sidecar content. Cleanup always reads original bytes again, rather than trusting cache entries.
- A run folder permits one operation at a time. If a process is interrupted, check that it has stopped before removing a stale `.operation.lock` file. Rerun a scan to reuse successfully cached tasks; completed per-file quarantine operations can be recognized from verified backups on reapplication.
- Optional package failures and unsupported formats are recorded. Empty extraction is never treated as duplicate evidence.
- Old binary Office formats, native cloud shortcut stubs, scanned PDFs, video visual similarity and OCR-based photo classification need additional converters or future workers. Ordinary full-file hashes still cover readable unsupported files.
- This is an audit tool, not a malware sandbox. Use an isolated environment for untrusted documents and keep parser dependencies updated.

Tests

```
python -m unittest discover -s tests -v
```

The suite uses temporary synthetic data. It exercises cross-year duplicates, sidecar interpretation, cache reuse, filters, Office extraction, metadata-only image differences, changed-file protection, plan binding, keeper protection, quarantine, restoration, hardlinks, symlinks and permanent-purge gates. It never accesses your real Google library. OCR, Ollama, Docker and Google cloud operations require separate environment-specific verification.

Technical documents

- Architecture and agent contracts
- Illustrated architecture
- Printable architecture and operations guide
- Mermaid diagram source
- Release notes

Primary references: Google Photos API changes, Google export guidance, pypdf extraction limits, TF-IDF, MiniBatchKMeans, and Ollama chat API.

Technical Architecture Appendix

Purpose and boundary

The system converts an exported personal library into an accountable inventory, duplicate evidence and subject reading packs before enabling a reviewed cleanup workflow. It runs as a local Python application. A sandbox or container can host the same program when the operator provides explicit input and output mounts.

The source of truth for analysis is the complete local file, not a thumbnail or filename. The source of truth for cloud identity is metadata supplied by the export, when available. Local byte sizes never establish cloud quota savings. No worker has Google account credentials or cloud deletion methods.

Component layout

Editable Mermaid diagrams are included in the accompanying architecture files.

Agent responsibility and permission contract

Agent	Input	Output	Permission
Coordinator	Configured roots and settings	Year/format tasks and run artifacts	Read sources; write run folder
Inventory	Filesystem metadata and small JSON sidecars	Canonical paths, IDs, year basis and metadata associations	Read sources
Year specialist	Files assigned to one year and modality	Complete hash plus extraction evidence	Read sources only
Document	Supported document bytes	Labeled text units, links, completeness status	No execution or macros
Photo	Image bytes and EXIF	Dimensions, oriented pixels, pHash, optional OCR	No source edits
Duplicate	All selected records across all years	Separate evidence classes	No cleanup tools
Subject	Non-empty extracted text	Statistical subject suggestions	No deletion decisions
Consolidation	Topic groups and text units	Markdown pack and block source map	Write derived output only
Cleanup	Catalog-bound approved plan	Verified quarantine and journal	Local approved files only
Restore	Original catalog and verified backup	Restored local files without overwrites	No cloud operations
Purge	Separate approved purge plan and confirmation	Permanent removal of validated quarantine copies	Never similarity-only candidates

DocumentAgent and PhotoAgent are specialized readers invoked by YearAgent. Hash-only work is performed by YearAgent for other file types. ThreadPoolExecutor provides concurrent local execution; workers return data to the coordinator, which serializes SQLite writes. No SQLite connection is shared between worker threads.

Scheduling and year semantics

Discovery produces canonical file paths and one stable ID per absolute path. Overlapping input roots do not create duplicate path records. The inventory stage indexes recognized Takeout sidecars, then creates a task for each selected `(year, specialist)` pair. Multiple tasks execute up to the configured worker limit. Completion order does not determine report ordering.

Year priority is Takeout `photoTakenTime` in UTC, EXIF capture date, a year inferred from filename/folder, then filesystem modification time in UTC. The final year and basis are preserved per item. Photos with only EXIF dates are classified before a year filter is applied. Year inference does not claim that a download timestamp is the original capture date.

A year worker performs sequential file reads within its queue. This limits simultaneous full-size file loads but means a single large year/modality queue can use only one worker. This implementation is intentionally a local worker pool; moving it to a distributed cluster would require a shared object store, durable task broker, lease/retry protocol and database writer service.

Pipeline states

Editable Mermaid diagrams are included in the accompanying architecture files.

File status and extraction status are distinct. `status=indexed` establishes a stable complete hash; the document may still be `unsupported_format`, `missing_dependency`, `truncated`, `needs_ocr_or_empty` or `extraction_error`. Similar-text evidence excludes incomplete extraction. Full-byte comparisons remain available for readable unsupported formats.

Data contracts

`catalog.json` is versioned and contains `run_id`, `source_roots`, settings, item records, errors and filter exclusions. Item records include `id`, absolute `path`, `category`, `year`, `year_basis`, byte `size`, file `signature`, `sha256`, source metadata, `cloud_url`, and extraction results. A signature contains size, modification nanoseconds, change nanoseconds, device ID and inode ID.

The document result has `status`, `units` with a stable label and extracted text, `urls` and limitation notes. The photo result has decoding status, dimensions, EXIF date, oriented RGBA pixel hash, perceptual hash, optional OCR text and notes. The catalog intentionally contains private material and must be stored like the source library.

`duplicates.json` has separate collections for exact bytes, repeated export representations, equal decoded pixels, equal extracted text and visually similar pairs. Findings reference catalog IDs rather than editable paths. A SHA-256 match is full-file byte evidence; a pHash distance is a review signal with different meaning.

`events.jsonl` records scheduling and year-worker completion. `cache.sqlite3` stores per-file extraction results keyed by path, signature, pipeline version, configuration and associated sidecar data. `status.json` records completed publication. A failed run may leave a running status and a lock; the operator must verify the process has stopped before clearing that lock.

Exact duplicates and Takeout representations

Hash and size group the entire selected corpus, including cross-year matches. Files recognized as Takeout metadata are not cleanup candidates. If all members of a byte-identical group have the same non-empty Photos URL, the finding is classified as repeated export representation, and generated plans omit it. Multiple albums can reference one original cloud item.

When cloud URLs are missing or differ, exact groups are local byte duplicates only. Their reported redundant bytes are an upper bound for local copies, not confirmed storage savings; hardlinks and cloud quota exemptions can change physical or billed storage. The planner excludes a removal if it and the retained copy share filesystem identity.

Subject grouping and source provenance

TF-IDF represents extracted text with word and two-word features. MiniBatchKMeans suggests an explicit or automatically chosen number of topics. Terms with high cluster weights label a topic; they are not a validated semantic taxonomy. Subject grouping is global across the selected years.

The consolidation worker emits one current-run reading pack per group. Each distinct normalized text block appears once, with every contributing file ID and page/slide/block reference. A JSON source map stores the same block provenance. Deduplicating a text block in a derived pack does not remove that block or its file from any original document.

Optional Ollama summaries receive an instruction boundary and at most the first 50,000 characters of a pack. They have no tool channel or cleanup capability. A prompt injection inside an original remains untrusted source text. Summary quality still requires human fact checking and cannot authorize cleanup.

Cleanup transaction sequence

Editable Mermaid diagrams are included in the accompanying architecture files.

A single operation lock prevents scan and cleanup operations from sharing a run folder concurrently. The entire approved set is preflighted before source changes. Each file then has a prepared event, verified backup, source revalidation and final journal event. Already-created correct backups support reapplication after interruption. The sequence is not atomic across the entire library: a disk failure can leave a completed prefix of operations, with backups and journal evidence.

Paths come from the digest-bound catalog, not arbitrary edited plan fields. Symlink and junction paths, source/run/quarantine nesting, missing keepers, stale hashes, duplicate removal IDs and keeper-removal conflicts are rejected. Permanent purge requires a second approved plan and an exact confirmation string. All destructive behavior is local and opt-in.

Deployment and optional MCP extension

Windows and WSL use the same Python package. Docker adds a non-root runtime and read-only source mount for analysis. The output directory must be persistent and writable. Cleanup and restore must run in the same path namespace as the catalog; a host cannot directly execute a plan containing container-only `/input` paths.

No MCP server is required or shipped. A future adapter can map configured run IDs to read-only status, summary and review-query tools. Authentication, allowed roots and operator policy belong outside the model. A tool adapter should never expose an unrestricted filesystem path or a permanent-purge endpoint to automatic model calls.

Google Photos API access is limited to app-created media, so MCP is not a substitute for a complete Takeout export. Cloud deletion is a distinct future connector requiring explicit supported capabilities, verified cloud identity, account checks, a review record and appropriate authorization. This package does not implement that connector or browser deletion automation.

Scaling and release verification

Full hashing is linear in input byte volume. Cache hits reduce repeat extraction. Similar-image matching batches queries and bounds nearest-neighbor outputs, but still has substantial pairwise computation. Subject grouping and extracted content are memory-resident. Default limits are appropriate for a laptop trial, not an unbounded corpus.

Tests cover positive and negative cleanup paths with temporary data, plus cross-year analysis and supported parsers. Real cloud deletion is absent by design. Docker runtime, optional OCR and Ollama require independent environment checks before relying on them. The illustrated guide and source code identify these limits rather than treating missing results as proof that data is absent.